Informe de Sprint 5

Antonio de Benito, Pablo Berástegui y Gonzalo García

Período: 25/03/2026 – 05/04/2026

Proyecto

TradeData – Análisis de Datos de Trading de Criptomonedas

1. Objetivo del Sprint

En este sprint se han abordado las historias de usuario **HU-6** y **HU-7**, ambas orientadas al procesamiento de datos **en tiempo real**. El objetivo principal ha sido construir una primera tubería de streaming capaz de adquirir cotizaciones en directo desde **Binance**, publicarlas en un **topic de Kafka** y, a continuación, procesarlas mediante **Spark Structured Streaming** para calcular el indicador **VWAP** en ventanas temporales de 5 minutos.

La primera parte del sprint se centra en la **ingesta de eventos de mercado**: capturar únicamente velas cerradas, serializar la información relevante y dejarla disponible en Kafka con el formato requerido. La segunda parte se centra en el **procesamiento continuo del flujo**, agregando los mensajes recibidos para obtener una métrica de análisis técnico en streaming y publicar los resultados en un segundo topic.

El repositorio del sprint se puede consultar en: datos-grandes.

2. Historias de Usuario Incluidas

2.1. Historia de usuario 6

Como: Manager del proyecto
Quiero: Que las consultoras:

- Creen un topic de Kafka correspondiente a su grupo, con una partición y factor de replicación 1 (ver tabla adjunta).
- Realicen la adquisición de datos en tiempo real de Binance de su criptomoneda y publiquen mensajes en el topic de Kafka correspondiente con la información obtenida:
  - Clave: símbolo de la criptomoneda.
  - Valor: formato JSON.
  - Timestamp: expresado en milisegundos (ms).

Ejemplo de mensaje:

```
{
  "symbol": "BTCUSDT",
  "@timestamp": "2026-03-09T11:21:00Z",
  "close": 67396.99,
  "volume": 19.14504
}
```

Para: Almacenar la información en tiempo real

2.1.1. Análisis

La HU-6 introduce una arquitectura de **ingesta streaming** basada en **Binance** y **Kafka**. El requisito fundamental no consiste solo en escuchar datos en tiempo real, sino en hacerlo de manera controlada para que los mensajes publicados sean consistentes y útiles para fases posteriores de procesamiento.

En particular, la historia de usuario exige que:

- exista un **topic de entrada** específico para el grupo;
- la información se obtenga directamente desde **Binance**;
- cada mensaje publicado tenga como **clave** el símbolo de la criptomoneda;
- el **valor** esté codificado en **JSON**;
- el evento se publique únicamente cuando la **vela esté cerrada**;
- el timestamp utilizado sea el proporcionado por Binance, expresado en milisegundos y reflejado también en el campo `@timestamp` en formato UTC.

Para satisfacer estas condiciones, se implementó un productor en Python apoyado en `python-binance` y `kafka-python`. El flujo seguido por el productor es el siguiente:

1. Se establece una conexión con el flujo de velas de Binance para la criptomoneda asignada.
2. Se reciben eventos de tipo *kline* con frecuencia de 1 minuto.
3. Se comprueba el campo que indica si la vela está definitivamente cerrada.
4. Solo en ese caso se construye el mensaje final y se envía al topic Kafka.

Este filtrado es importante porque durante la formación de la vela Binance emite actualizaciones intermedias. Si se publicaran todas ellas, el topic contendría información repetida y todavía no consolidada. Al publicar únicamente la vela cerrada, cada mensaje representa un dato final de mercado y puede utilizarse de forma fiable en el procesamiento posterior.

En nuestro caso, el flujo ha quedado configurado con:

- **Topic de entrada**: `imat3a_ETH`
- **Criptomoneda**: `ETHUSDT`
- **Particiones**: 1

- **Factor de replicación**: 1
- **Bootstrap servers**: `51.49.235.244:9092`

Además, el mensaje publicado contiene los campos mínimos necesarios para el análisis posterior:

- `symbol`: símbolo de la criptomoneda.
- `@timestamp`: instante de cierre de la vela en formato UTC.
- `close`: precio de cierre.
- `volume`: volumen negociado durante la vela.

2.1.2. Implementación

A continuación se incluye el código principal del productor encargado de adquirir datos desde Binance y publicarlos en Kafka.

```
# -*- coding: utf-8 -*-
import json
from kafka import KafkaProducer
from binance import Client
from binance import ThreadedWebsocketManager

from datetime import datetime

BOOTSTRAP_SERVERS="51.49.235.244:9092"
USERNAME="kafka_client"
PASSWORD="88b8a35dca1a04da57dc5f3e"
TOPIC="imat3a_ETH"
KEY = "ETHUSDT"

producer = KafkaProducer(
    bootstrap_servers=BOOTSTRAP_SERVERS,
    security_protocol="SASL_PLAINTEXT",
    sasl_mechanism="PLAIN",
    sasl_plain_username=USERNAME,
    sasl_plain_password=PASSWORD,
    value_serializer=lambda v: json.dumps(v).encode("utf-8"),
    key_serializer=lambda v: v.encode("utf-8")
)

def ms2utc(ms: int) -> str:
    return datetime.fromtimestamp(ms / 1000).strftime('%Y-%m-%dT%H:%M:%S.%fZ'
    )

def handle_kline(msg):

    global producer

    k = msg['k']
    if k['x']:
```

```
        key = k['s']
        value = {
            "symbol": key,
            "@timestamp": ms2utc(k['T']),
            "close": k['c'],
            "volume": k['v']
        }
        timestamp = k['T']
        producer.send(TOPIC, key=key, value=value, timestamp_ms=timestamp)


def main() -> None:

    # Crea el KafkaProducer

    global producer
    print("Creando hilo...")
    twm = ThreadedWebsocketManager()
    twm.start()
    print("Arrancando hilo...")
    while True:
        twm.start_kline_socket(
            symbol=KEY,
            interval=Client.KLINE_INTERVAL_1MINUTE,
            callback=handle_kline
        )

        if input("Pulsa ENTER para salir\n") == "":
            break
    print("Parando hilo...")
    twm.stop()

    producer.flush()
    producer.close()

if __name__ == "__main__":
    main()
```

Desde el punto de vista de implementación, conviene remarcar tres decisiones:

- se utiliza **websocket** para recibir datos en tiempo real sin necesidad de hacer consultas periódicas;
- se emplea el campo `k['x']` para garantizar que solo se emiten velas cerradas;
- el timestamp original de Binance se preserva tanto en el `timestamp_ms` del envío a Kafka como en el campo textual `@timestamp`.

2.1.3. Evidencias

Las evidencias que deben incluirse en esta sección tienen que demostrar dos aspectos: que el productor estaba enviando mensajes correctamente al topic de entrada y que el

consumidor podía leer esos mensajes con la estructura esperada.

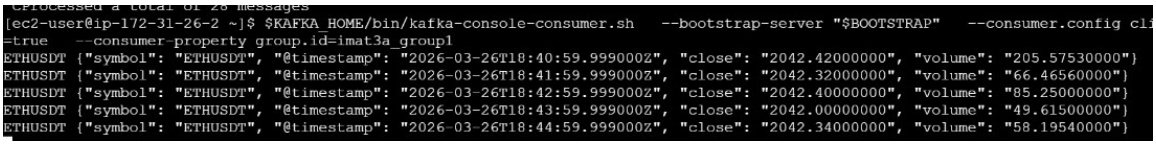

Figura 1: Captura 1. Consumer básico leyendo mensajes del topic de entrada

A partir de estas capturas debe verificarse que:

- los mensajes llegan al topic correcto;
- la clave coincide con el símbolo de la criptomoneda;
- el contenido JSON contiene `symbol`, `@timestamp`, `close` y `volume`;
- los mensajes se corresponden con cierres reales de vela y no con actualizaciones parciales.

2.2. Historia de usuario 7

Como: Manager del proyecto
Quiero: Que las consultoras:

- Desarrollen un sistema de análisis en tiempo real utilizando Spark Structured Streaming que, a partir de los datos de cotización de criptomonedas almacenados en el topic correspondiente a cada grupo (ej. `imat3a-BTC`), calcule el indicador VWAP por ventanas de 5 minutos y deje los resultados en el topic de salida (ej. `imat3a-BTC-VWAP`).
  - Clave: símbolo de la criptomoneda.
  - Valor: formato JSON.

Ejemplo de mensaje:

```
{
  "window_start": "2026-03-16T14:20:00.000Z",
  "window_end": "2026-03-16T14:25:00.000Z",
  "symbol": "BTCUSDT",
  "vwap": 74229.99
}
```

VWAP (Volume Weighted Average Price):

- Fórmula: $\text{VWAP} = \frac{\sum(\text{precio} \times \text{volumen})}{\sum(\text{volumen})}$.
- Qué mide:
  - Precio "justo" ponderado por volumen.
  - Reduce el ruido de precios con poco volumen.

Para: Almacenar información calculada en tiempo real

2.2.1. Análisis

La HU-7 extiende la ingesta desarrollada en la HU-6 con una fase de **procesamiento continuo**. El objetivo ya no es únicamente almacenar eventos de mercado, sino transformarlos en una métrica agregada que permita interpretar mejor la evolución del activo en tiempo real.

Para ello se ha diseñado un job en **Spark Structured Streaming** que consume de forma continua el topic de entrada, interpreta el JSON recibido y calcula el indicador **VWAP** en ventanas de 5 minutos.

El proceso lógico seguido por esta aplicación es el siguiente:

1. Lectura continua desde el topic Kafka de entrada.
2. Conversión de la clave y del valor desde formato binario a texto.
3. Parseo del valor JSON con un esquema explícito.
4. Conversión del campo temporal a tipo `timestamp`.
5. Agrupación por símbolo y ventana temporal de 5 minutos.
6. Cálculo del VWAP mediante la expresión:

$$VWAP = \frac{\sum(\text{close} \cdot \text{volume})}{\sum(\text{volume})}$$

7. Serialización del resultado en JSON y publicación en el topic de salida.

La elección de **Spark Structured Streaming** permite mantener un flujo continuo de lectura, cálculo y escritura sin necesidad de lanzar jobs batch independientes. De este modo, el sistema reacciona automáticamente a la llegada de nuevos eventos en Kafka.

En nuestra implementación, los elementos principales del flujo han sido:

- **Topic de entrada**: `imat3a_ETH`
- **Topic de salida**: `imat3a_ETH_VWAP`
- **Ventana de agregación**: 5 minutos
- **Criptomoneda**: `ETH`
- **Modo de salida**: `update`
- **Checkpoint**: `eth_vwap_simple_v1`

Desde el punto de vista analítico, el uso del VWAP resulta adecuado porque no promedia todos los precios por igual, sino que da más peso a aquellos intervalos con mayor volumen negociado. Por ello, ofrece una referencia más representativa del precio efectivo de negociación durante la ventana considerada.

```
from pyspark.sql import SparkSession
from pyspark.sql.functions import col, from_json, window, sum as _sum, struct,
    to_json, to_timestamp
from pyspark.sql.types import StructType, StructField, StringType, DoubleType
```

```
spark = SparkSession.builder \
    .appName("Calculo_VWAP_ETH") \
    .config("spark.jars.packages", "org.apache.spark:spark-sql-kafka-0-10_2
   .12:3.3.0") \
    .getOrCreate()

# Reducimos el spam en la consola
spark.sparkContext.setLogLevel("WARN")

BOOTSTRAP_SERVERS = "51.49.235.244:9092"
TOPIC_IN = "imat3a_ETH"
TOPIC_OUT = "imat3a_ETH_VWAP"
CHECKPOINT_LOCATION = "/tmp/spark_checkpoints/eth_vwap_simple_v1"

KAFKA_SASL_JAAS_CONFIG = (
    'org.apache.kafka.common.security.plain.PlainLoginModule required '
    'username="kafka_client" password="88b8a35dca1a04da57dc5f3e";'
)

schema_entrada = StructType([
    StructField("symbol", StringType(), True),
    StructField("@timestamp", StringType(), True),
    StructField("close", StringType(), True),
    StructField("volume", StringType(), True)
])

def main():

    df_crudo = spark.readStream \
        .format("kafka") \
        .option("kafka.bootstrap.servers", BOOTSTRAP_SERVERS) \
        .option("subscribe", TOPIC_IN) \
        .option("startingOffsets", "latest") \
        .option("kafka.security.protocol", "SASL_PLAINTEXT") \
        .option("kafka.sasl.mechanism", "PLAIN") \
        .option("kafka.sasl.jaas.config", KAFKA_SASL_JAAS_CONFIG) \
        .load()

    df_json = df_crudo.selectExpr("CAST(value AS STRING) as json_val") \
        .select(from_json(col("json_val"), schema_entrada).alias("data")) \
        .select("data.*") \
        .withColumn("event_timestamp", to_timestamp(col("@timestamp"), "yyyy-MM
   -dd'T'HH:mm:ssX"))

    df_clean = df_json.filter(
        col("symbol").isNotNull() &
        col("event_timestamp").isNotNull() &
        (col("volume").cast("double") > 0)
    )

    df_precalc = df_clean.withColumn("precio_x_volumen", col("close").cast("
```

```
    double") * col("volume").cast("double")) \
                        .withColumn("volume_num", col("volume").cast("double")
    )

    df_agrupado = df_precalc.withWatermark("event_timestamp", "2 minutes") \
        .groupBy(
            window(col("event_timestamp"), "5 minutes"),
            col("symbol")
        ).agg(
            _sum("precio_x_volumen").alias("sum_pv"),
            _sum("volume_num").alias("sum_v")
        )

    # Calculamos el VWAP final: Sum(P*V) / Sum(V)
    df_vwap = df_agrupado.withColumn("vwap", col("sum_pv") / col("sum_v"))

    df_salida = df_vwap.select(
        col("symbol").alias("key")
        to_json(struct(
            col("window.start").alias("window_start"),
            col("window.end").alias("window_end"),
            col("symbol"),
            col("vwap")
        )).alias("value") # El valor de Kafka es el JSON completo
    )

    query = df_salida.writeStream \
        .format("kafka") \
        .option("kafka.bootstrap.servers", BOOTSTRAP_SERVERS) \
        .option("topic", TOPIC_OUT) \
        .option("kafka.security.protocol", "SASL_PLAINTEXT") \
        .option("kafka.sasl.mechanism", "PLAIN") \
        .option("kafka.sasl.jaas.config", KAFKA_SASL_JAAS_CONFIG) \
        .option("checkpointLocation", CHECKPOINT_LOCATION) \
        .outputMode("update") \
        .trigger(processingTime="1 minute") \
        .start()

    query.awaitTermination()

if __name__ == "__main__":
    main()
job.commit()
```

2.2.2. Evidencias

Las evidencias de esta historia de usuario demuestran que el job de streaming estaba consumiendo correctamente los mensajes del topic de entrada, calculando el VWAP y publicando los resultados agregados en el topic de salida.

Figura 2: Consumer de VWAP

Como se puede entender en el flujo del código, cada vez que se publica un mensaje en el topic básico, se toman los valores dentro de la ventana de los últimos 5 minutos, de tal manera que cada minuto obtenemos el VWAP de los últimos 5 minutos periódicamente.

3. Arquitectura del Sprint

La arquitectura del sprint queda definida como una tubería de streaming en dos etapas: **ingesta** y **procesamiento**. La primera etapa captura datos de Binance y los deja disponibles en Kafka. La segunda etapa consume esos datos y genera un flujo derivado con indicadores calculados en tiempo real.

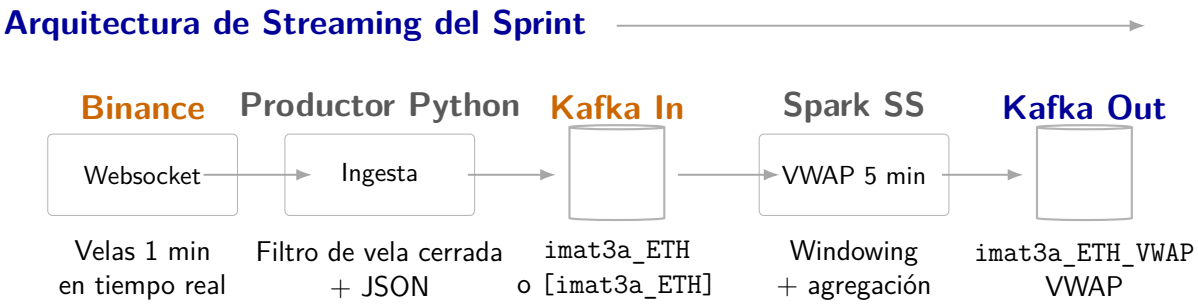

Figura 3: Arquitectura del sprint

4. Validación y Pruebas

En la HU-6 se validó que el productor recibía **cotizaciones en tiempo real desde Binance** y publicaba únicamente velas cerradas en el topic de entrada, con la clave y la estructura JSON esperadas. La verificación se realizó observando el consumidor de Kafka y confirmando que los mensajes llegaban de forma consistente y con los campos requeridos para el procesamiento posterior.

En la HU-7 se validó que Spark Structured Streaming consumía correctamente el topic de entrada, parseaba el evento temporal, calculaba el **VWAP en ventanas de 5 minutos** y publicaba los resultados en el topic de salida con el formato esperado. La prueba de funcionamiento se completó manteniendo el pipeline activo durante varios ciclos para comprobar la generación continua de resultados agregados.

5. Conclusiones del Sprint

Este sprint ha supuesto la transición del proyecto desde un enfoque puramente batch hacia una arquitectura con capacidad de **procesamiento en tiempo real**. El resultado es una

tubería funcional que permite capturar cotizaciones desde Binance, publicarlas en Kafka y procesarlas automáticamente con Spark Structured Streaming.

En concreto:

- se ha implementado una fase de **ingesta en tiempo real** robusta, publicando únicamente datos cerrados y consistentes;
- se ha añadido una fase de **procesamiento streaming** capaz de calcular el indicador VWAP por ventanas temporales;
- se ha dejado preparada una arquitectura extensible para incorporar nuevos indicadores, nuevos consumidores o sistemas de monitorización;
- se ha validado el funcionamiento del flujo completo mediante consumidores en Kafka, tanto para el topic de entrada como para el topic de salida.

Como resultado, el proyecto ya no solo almacena y transforma datos históricos, sino que también es capaz de reaccionar ante eventos de mercado conforme estos se producen, sentando la base para futuras capacidades de analítica en tiempo real.